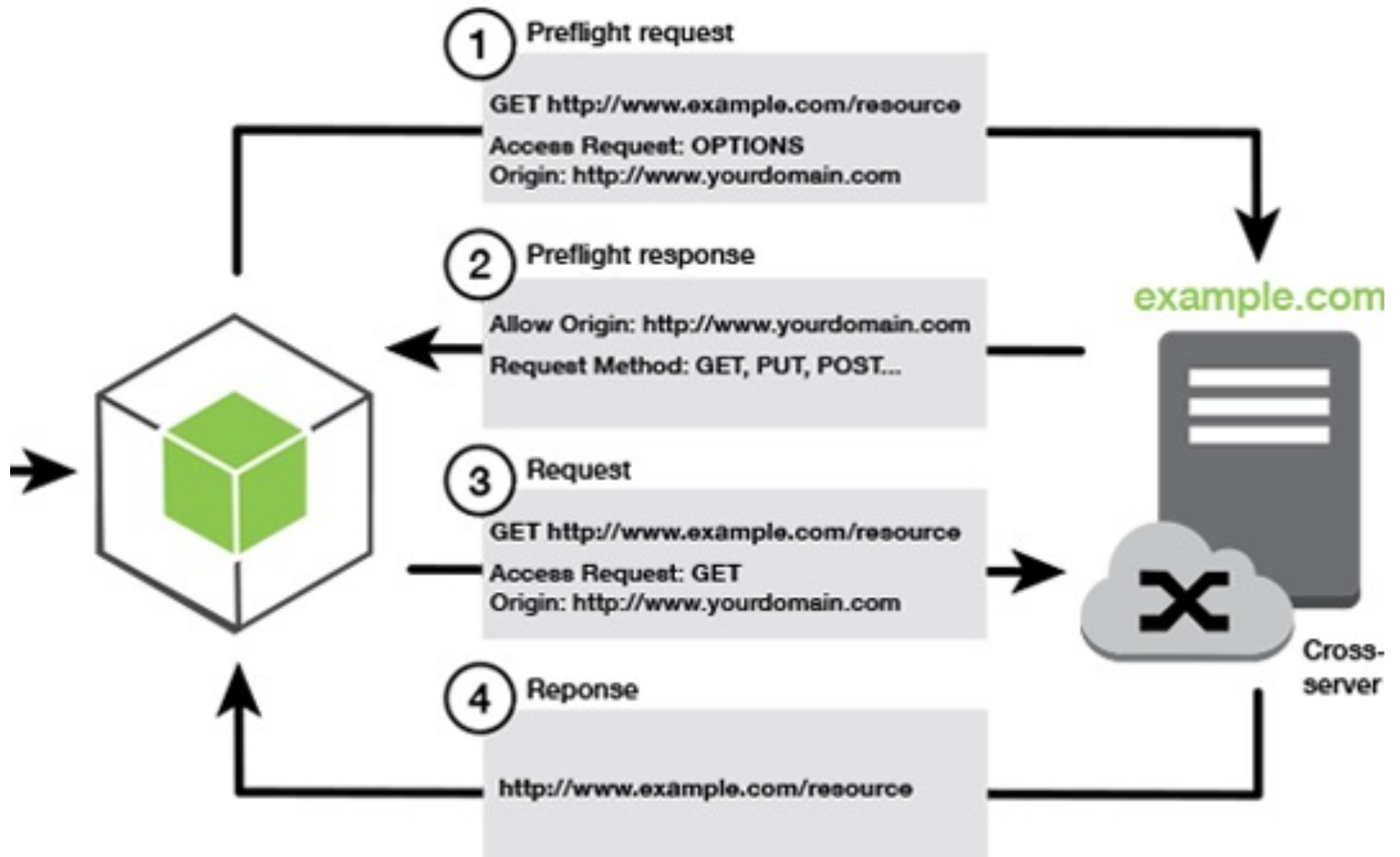




## Cross-Origin Resource Sharing (CORS) – I

L'**origine** di un'applicazione **client-side** è il **Server HTTP** che l'ha fornita. Si parla di **cross-origin resource sharing** quando un'applicazione **client-side** vuole effettuare una **fetch** da una server-side app che risiede su un'**origine** diversa.

Quando si verifica una richiesta di questo tipo, il browser, prima di far partire l'effettiva **fetch** che compare nel codice dell'app client-side, effettua al server una richiesta detta **preflight**, che usa l'HTTP method OPTIONS. La **preflight request** è una sorta di test preliminare che permette al browser di capire se la richiesta effettiva che seguirà potrà essere recepita dall'app server-side.

Nella preflight request, il browser descrive la richiesta effettiva che vorrebbe effettuare attraverso tre **header**:

- **Access-Control-Request-Method**: quale sarà l'HTTP method della richiesta
- **Origin**: qual è l'origine dell'applicazione client-side che effettuerà la richiesta
- **Access-Control-Request-Headers**: specifica gli HTTP header della richiesta che verrà effettuata

L'HTTP Server risponde a una **preflight request** fornendo informazioni su quali sono gli HTTP method, gli header e le origini permesse. Il browser sa a questo punto se la richiesta che sta per mandare verrà accettata o meno e quali header dovrà inserire nella richiesta perché possa essere presa in considerazione dal server.

**Se la preflight request viene respinta, è il browser stesso a bloccare la richiesta originale che l'applicazione client-side aveva generato.**

Se la mandasse avanti, sarebbe l'applicazione server-side a rispondere con un errore (in base a qual è stata la violazione delle regole di CORS; potrebbe essere 405 Method Not Allowed o 403 Forbidden). Gli header utilizzati dall'app server-side nella risposta alla preflight request per esprimere i propri vincoli sono:

- **Access-Control-Allow-Methods**: l'elenco degli HTTP method permessi
- **Access-Control-Allow-Headers**: l'elenco degli header permessi
- **Access-Control-Allow-Origin**: l'elenco delle origini permesse
- **Access-Control-Allow-Credentials**: se è permesso l'invio di credenziali (ciò si riferisce essenzialmente ai Cookie di sessione)

## Cross-Origin Resource Sharing (CORS) – II

Per default, i Server HTTP bloccano le richieste cross-origin (adottano cioè una politica noCORS). Questo spesso costituisce un problema in fase di sviluppo dell'applicazione, quando, come accade a noi, abbiamo due diversi Server HTTP: uno fornisce l'applicazione client-side, l'altro è di fatto «embedded» all'interno della nostra applicazione server-side. Anche se girano sullo stesso host (**localhost**) sono in esecuzione su due porte diverse. Diventa dunque importante capire come permettere almeno parzialmente le richieste CORS.

### Primo approccio semplificato

Un modo molto semplice, che è consigliato usare **solo ed esclusivamente in fase di sviluppo su localhost**, consiste nell'includere l'annotazione `@CrossOrigin` per il **controller** o anche per quei solo **request handler** per cui vogliamo permettere le richieste cross-origin.

In fase di sviluppo, questo è probabilmente vero per **tutte** le richieste, dunque annoteremo con `@CrossOrigin` tutti i nostri controller.

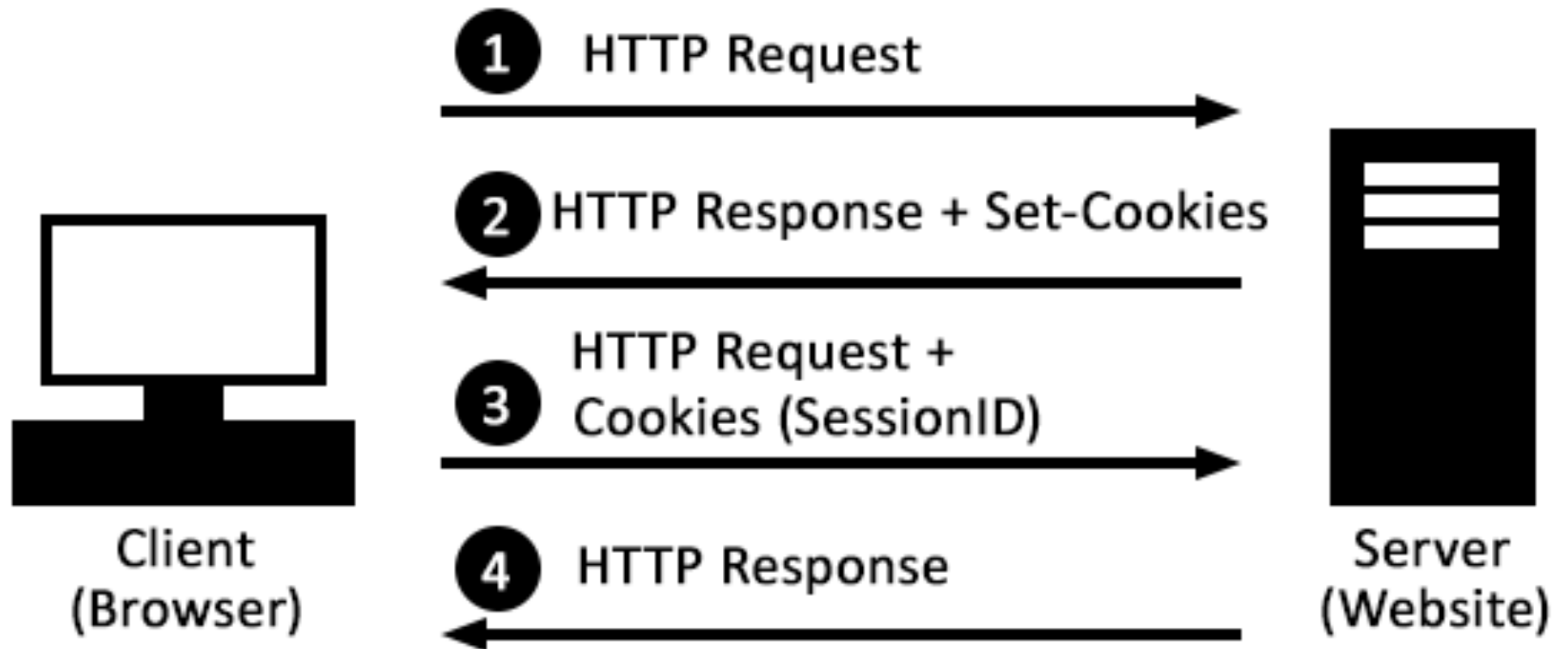
**Attenzione:** le richieste HTTP che non corrispondono a nessuno dei nostri handler/controller, saranno ancora soggette alla politica **noCORS**. Quindi a prescindere dal fatto che, se nessun handler le gestisce, non sono richieste valide, esse verranno «bloccate» in fase di **preflight** ancora prima che si possa stabilire se sono valide o meno.

Questo tipo di uso dell'annotazione `@CrossOrigin` implica che, quando la richiesta è effettivamente diretta a uno dei nostri controlli, essa venga permessa **indipendentemente** dall'origine, dall'HTTP Method o dagli header utilizzati. In pratica è una sorta di salvacondotto universale. Solo l'invio di credenziali viene ancora bloccato.

- **Access-Control-Allow-Methods:** l'insieme degli HTTP Method a cui risponde il **controller**
- **Access-Control-Allow-Headers:** qualsiasi
- **Access-Control-Allow-Origin:** qualsiasi
- **Access-Control-Allow-Credentials:** non permesse

# Sessioni

## Implementazione tramite Cookie



## Il concetto di «sessione»

Quando c'è un'interazione prolungata fra due «parti» (che siano un utente e un'applicazione, un client e un server, due applicazioni peer-to-peer...) diventa rilevante il concetto di **sessione**: l'idea cioè che la «conversazione» abbia uno stato, uno storico, e che in ogni passo della conversazione si possa fare riferimento a quanto già detto e già fatto, senza dover ogni volta ripetere o ricapitolare i passi precedenti.

Questo pone un problema nel contesto delle «conversazioni HTTP», per due ragioni:

- Il protocollo HTTP è di per sé «stateless», non prevede un'idea di stato dello scambio. Ogni transazione HTTP (richiesta + risposta) è indipendente dalle altre. L'unico modo di «simulare» l'esistenza di uno stato è ad ogni passo della conversazione riportare tutte le informazioni rilevanti scambiate in precedenza.
- Un server HTTP gestisce multiple conversazioni HTTP con diversi client, quindi, a meno che il client non si renda in qualche modo riconoscibile, non è possibile per il server sapere «a quale punto» della conversazione si trova, semplicemente perché sarà in punti diversi a seconda dell'interlocutore.

Quindi è necessario «costruire» uno stratagemma **al di sopra** del protocollo HTTP per permettere a client e server in primis di sapere che stanno avendo una conversazione prolungata, e in secondo luogo per mantenere una sorta di «stato» di tale conversazione.

## Sessioni tramite cookie

Uno dei modi più semplici per tener traccia di una sessione tramite il protocollo HTTP è attraverso l'uso dei **cookie**. Un cookie è un **blocco di dati** che il server manda al client nel corso del **primo scambio** della loro conversazione; ad ogni successivo scambio, il client aggiungerà il cookie alla propria richiesta, in modo che il server identifichi il particolare thread di conversazione che sta avendo con quel client.

Il server può memorizzare informazioni all'interno del cookie, in modo da ritrovarle nei passi successivi: questo è un modo di mantenere lo stato della conversazione. Solitamente quello che in realtà accade è che nel cookie si memorizzano le informazioni necessarie a riconoscere l'utente e identificare univocamente la conversazione; lo stato della conversazione stessa viene conservato in memoria o nel DB, indicizzato a partire dall'id univoco della conversazione. In questo modo si evita di rendere troppo pesanti le richieste/risposte HTTP.

## Gestione della sessione in Spring Boot

Per attivare l'uso dei cookie di sessione in una applicazione server-side con Spring Boot dovremo innanzitutto configurare il file **application.properties** aggiungendo le seguenti impostazioni:

```
server.servlet.session.cookie.name=LAB_SESSION_COOKIE  
server.servlet.session.cookie.http-only=true  
server.servlet.session.cookie.secure=true
```

Il **nome** del cookie serve a identificarlo (come vedremo, i cookie memorizzati dal browser sono visibili negli strumenti di sviluppo).

Specifichiamo poi che si tratta di un cookie **http-only**, ossia che il browser e il codice dell'applicazione client-side non hanno accesso ai contenuti del cookie, e che il cookie deve essere **secure** ossia cifrato.

Per accedere al **cookie di sessione**, possiamo **iniettare** nei **request handler** l'oggetto HttpSession. Tramite esso possiamo:

- **Scrivere** nel cookie: session.setAttribute(key, value)
- **Leggere** dal cookie: session.getAttribute(key)
- **Invalidare** il cookie anche senza che il timeout sia scaduto: session.invalidate( )
- **Impostare** il timeout della sessione: session.setMaxInactiveInterval(secs)

Il **timeout** specifica per quanto tempo un cookie può rimanere inutilizzato (ossia: nessuna nuova richiesta con quel cookie giunge al server). Scaduto questo tempo il cookie non sarà più valido e dovrà esserne generato uno nuovo.

## La sessione e l'app client-side

Per far funzionare il meccanismo della sessione, l'applicazione client-side deve collaborare: deve infatti mandare indietro il cookie ad ogni richiesta HTTP che invia all'app server-side.

Si può pensare che questo dovrebbe essere il comportamento di default del browser (includere i cookie in ogni richiesta), e in effetti è così **solo quando la richiesta è same-origin!**

Quindi se siamo in una situazione **cross-origin**, per default le credenziali non vengono inviate al server.

Per ottenere che vengano inviate, dobbiamo specificare nelle **RequestOptions** della **fetch** l'opzione **credentials: "include"**

il valore di default di questa opzione è:

**credentials: "same-origin"**

## Cross-origin e uso di credenziali nell'app server-side

Se includiamo le credenziali nella richiesta HTTP, si pone un altro problema: l'annotazione **@CrossOrigin**, così com'è, non va più bene.

Il primo problema che si presenta è che la risposta alla **preflight request** deve includere l'header

- **Access-Control-Allow-Credentials: true**

Possiamo farlo con l'annotazione **@CrossOrigin**, specificandone la proprietà **allowCredentials**:

**@CrossOrigin(allowCredentials = true)**

Questo però porta ad un secondo problema: per ragioni di sicurezza, se le credenziali sono ammesse, il protocollo HTTP impedisce che l'applicazione server-side abiliti tutte le origini in modo acritico. In altre parole, l'impostazione di default di **@CrossOrigin** che pone:

- **Access-Control-Allow-Origin**: qualsiasi

non è più valida. È necessario elencare esplicitamente le origini ammesse, fornendo così una **whitelist**.

Ecco che l'annotazione diventa dunque qualcosa come:

**@CrossOrigin(origins = "http://localhost:5173", allowCredentials = "true")**

# Filtri

## Passare al vaglio le richieste HTTP

Un **filtro** in Spring Boot/Jakarta è una funzione attraverso cui passano le richieste HTTP prima di giungere ai **request handler**

```
@Component
@Order(1)
public class AuthFilter implements Filter {
    @Override
    public void doFilter(ServletRequest servletRequest,
                        ServletResponse servletResponse,
                        FilterChain filterChain)
        throws IOException, ServletException {

        HttpServletRequest request = (HttpServletRequest) servletRequest;
        HttpServletResponse res = (HttpServletResponse) servletResponse;

        ...
        filterChain.doFilter(servletRequest, servletResponse);

        ...
        res.sendError(HttpServletResponse.SC_FORBIDDEN);
    }
}
```

## Filtrare le richieste HTTP in Spring Boot / Jakarta

Quando c'è la necessità di eseguire operazioni preliminari sulle richieste HTTP che pervengono ad una app server-side, è utile implementare un **filtro**, ossia una funzione che verrà chiamata appunto per ogni richiesta HTTP prima di inoltrarla ad uno specifico **request handler**.

Uno degli scopi per cui può essere utilizzato un filtro è verificare che la richiesta HTTP soddisfi dei requisiti, in mancanza dei quali la richiesta verrà rifiutata. La funzione-filtro può quindi decidere se **inoltrare** la richiesta in quella che viene chiamata **filter chain** (catena dei filtri), e che culmina con l'invocazione di un **request handler**, oppure **respingere** la richiesta con un qualche **status code** di errore.

### Come specificare un filtro in Spring Boot

Un **filtro** in Spring Boot è

- una classe che implementa l'interfaccia **Filter**
- ed è annotata con `@Component`

Rispettando queste due condizioni si ottiene che Spring Boot includa il filtro in questione nella cosiddetta **filter chain**, ossia la sequenza di filtri (alcuni già presenti nel framework) invocati prima di passare effettivamente la richiesta allo **handler** opportuno. È possibile determinare l'ordine di esecuzione dei filtri usando l'annotazione `@Order(N)` dove N è il numero d'ordine.

L'interfaccia **Filter** richiede di implementare il metodo:

```
public void doFilter(ServletRequest servletRequest,  
                    ServletResponse servletResponse,  
                    FilterChain filterChain)  
    throws IOException, ServletException
```

Vediamone gli elementi costituenti.

## Il metodo doFilter

Il metodo **doFilter** prende i seguenti parametri:

- **un oggetto ServletRequest**: la richiesta HTTP nella sua forma «grezza», prima che Spring Boot la elabori per estrarne le diverse informazioni che potranno essere passate al request handler. La classe **ServletRequest** rappresenta una generica richiesta di rete; per ottenere le informazioni specifiche di una richiesta HTTP dobbiamo effettuare il **type casting** dell'oggetto al tipo **HttpServletRequest**.
- **un oggetto ServletResponse**: la risposta HTTP, anche in questo caso nella sua forma «grezza». È l'oggetto nel quale Spring Boot andrà a inserire le informazioni restituite dai request handler, per poi inviarle in risposta all'applicazione client-side. Anche in questo caso, per ottenere le funzionalità specifiche di una risposta HTTP, dobbiamo effettuare il **type casting** dell'oggetto al tipo **HttpServletResponse**.
- **un oggetto FilterChain**: è la catena di filtri a cui si dovranno inoltrare i due oggetti, **request** e **response**, per passare il testimone al prossimo filtro, o, se i filtri sono terminati, al request handler opportuno.

## Come inoltrare la richiesta HTTP nella catena di filtri

Per inoltrare la richiesta, «passiamo» gli oggetti ricevuti al prossimo attore nella catena, sarà sufficiente invocare:

```
filterChain.doFilter(request, response)
```

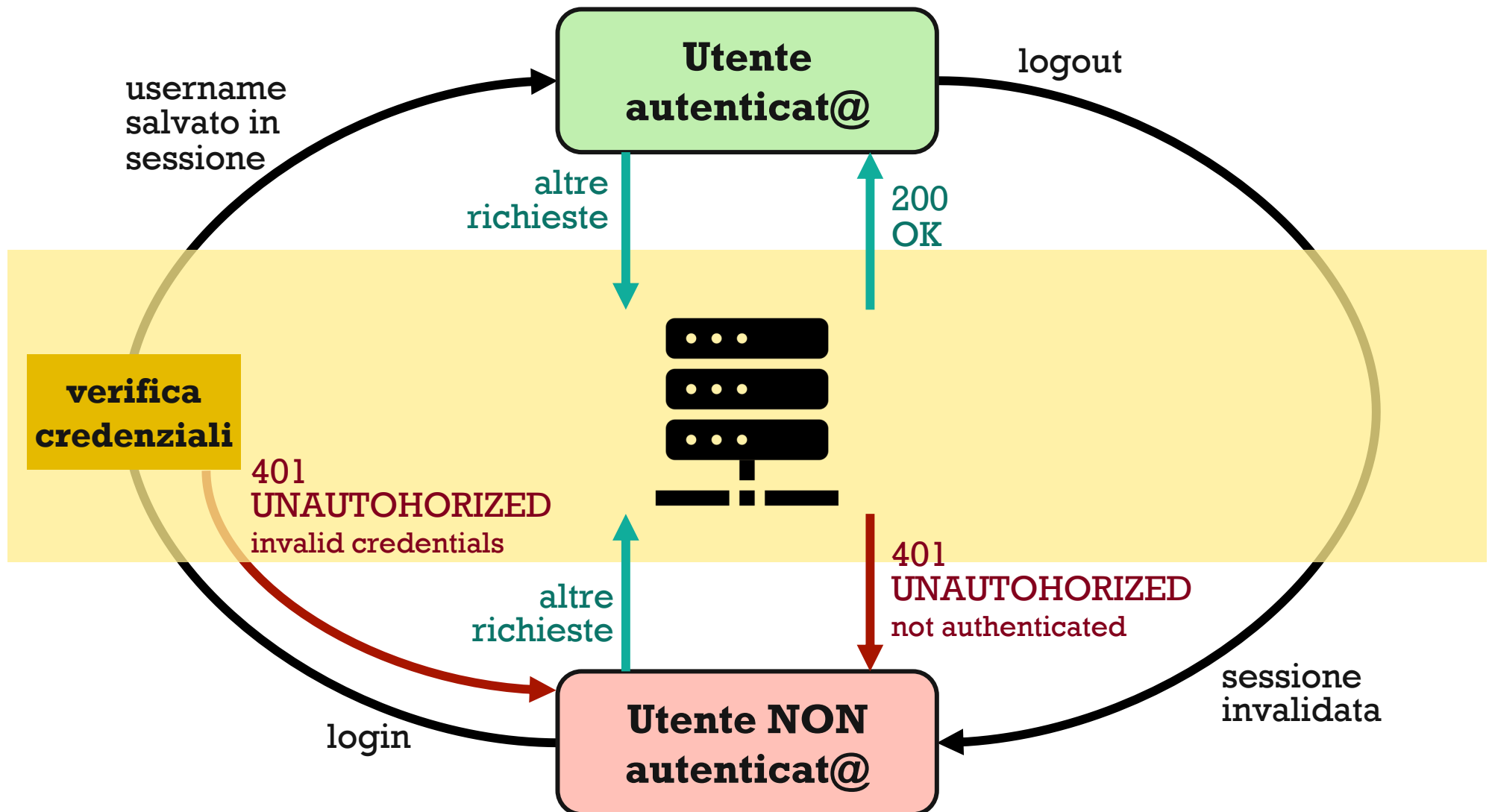
## Come bloccare la richiesta HTTP

Per bloccare la richiesta, anziché invocare la filter chain, ci limiteremo a utilizzare l'oggetto response per inviare al richiedente uno status code di errore:

```
response.sendError(HttpServletResponse.SC_XXX)
```

# Autenticazione

## Basic Auth con filtri e sessioni



## Basic Authentication con filtri e sessioni

Con gli ingredienti fin qui presi in considerazione, in particolare i filtri e le sessioni, è possibile «cucinare» un meccanismo di autenticazione di base.

In questo caso ne considereremo una versione particolarmente semplificata (e per nulla sicura) in cui username e password viaggiano in chiaro come **query parameters** nella richiesta HTTP. È possibile realizzare una Basic Auth un poco più sicura inserendo username e password nel **body** della richiesta (quindi, effettuando una POST anziché una GET) ed usando, per la connessione, il protocollo HTTPS anziché HTTP standard.

Per prima cosa, diciamo che distinguiamo un **utente autenticato** da uno **non autenticato** tramite un attributo (ad esempio lo username) salvato nella **sessione**.

Per una generica richiesta HTTP il filtro dovrà dunque verificare se la sessione contiene l'attributo in questione: se sì, farà passare la richiesta. Altrimenti risponderà con un codice 401 UNAUTHORIZED.

Fa eccezione a questo la richiesta di **login**, che deve essere fatta passare in ogni caso. La richiesta di login arriverà dunque al **request handler** il quale dovrà (tramite un opportuno @Service) determinare se le credenziali sono corrette. Se lo sono, potrà creare nella sessione l'attributo che «segna» la corretta autenticazione. In caso contrario risponderà anche in questo caso con un codice 401 UNAUTHORIZED.

Una richiesta di **logout** può dunque essere realizzata semplicemente **invalidando** la sessione.

Il server dovrà anche esporre una route, accessibile anche a utenti non autenticati, che permetta all'applicazione client-side di **verificare** lo stato corrente dell'autenticazione.

## Il ruolo dei framework server-side

Il ruolo principale dei framework nelle applicazioni server-side è quello di automatizzare parte dei compiti e rendere più uniformi le interfacce verso quello che abbiamo chiamato il «mondo esterno». Il framework si frappone fra il «core» dell'applicazione e l'ambiente di esecuzione, facendosi carico di parte dei compiti. Esattamente quali compiti siano gestiti dal framework, come siano gestiti e come sia possibile prevedere al loro interno l'esecuzione di codice personalizzato, è molto dipendente dal framework che si sceglie di usare.

La scelta di un Server HTTP (o Server Web) è fortemente legata alla scelta del linguaggio di programmazione che determina sostanzialmente la scelta del framework.

## Combo linguaggio/framework più usate nel web (2023):

- PHP/Laravel
- Python/Django
- Java/Spring
- Python/Flask
- Node.JS /Express

## La strana storia di PHP

Per decenni PHP è stato il principale linguaggio utilizzato per sviluppare applicazioni back-end. Si trattava però di applicazioni basate su un modello architetturale diverso (anche perché non erano stati ancora introdotti Ajax o JSON per lo scambio di dati; pur in evoluzione, il Web era ancora molto basato sull'idea di un Web Server che pubblica contenuti e un client che permette all'utente di fruirli. L'interattività dell'utente era molto limitata. Il back-end dunque sostanzialmente generava dinamicamente pagine HTML, e PHP è un linguaggio che semplifica molto questo compito.

Esempi tipici di questo tipo di applicativi di back-end sono Wordpress o Moodle, ossia i cosiddetti CMS (content management system).

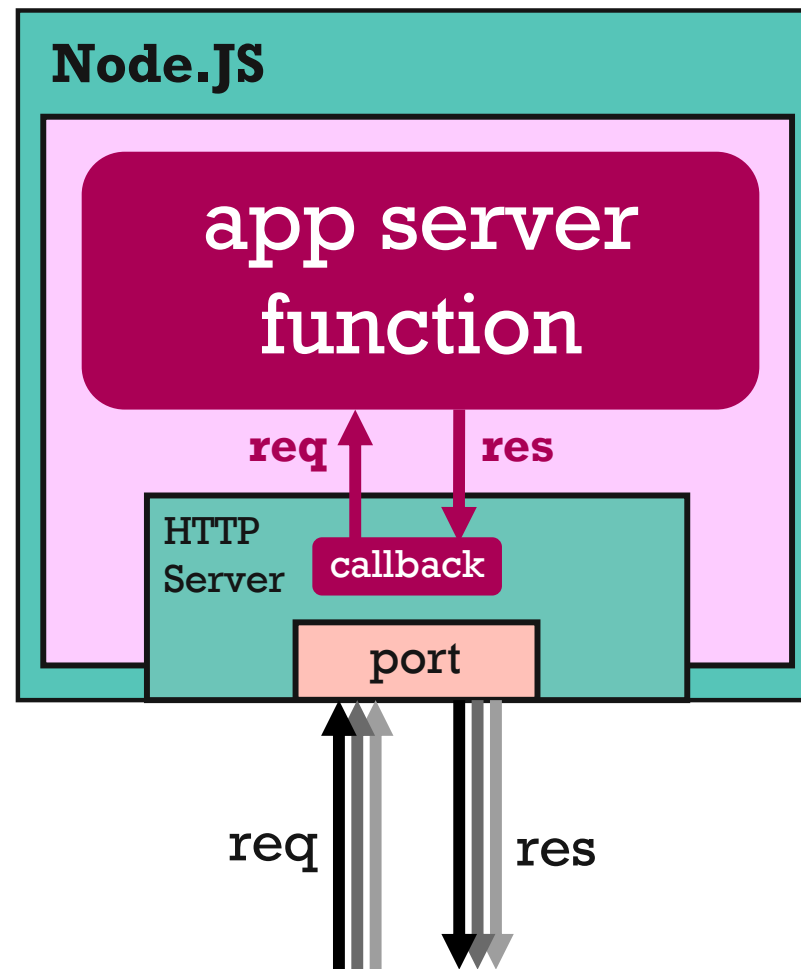
Queste applicazioni hanno una diffusione enorme e continuano ad essere aggiornate dagli sviluppatori, il che significa che **a oggi** PHP è ancora di gran lunga il linguaggio più utilizzato nel Web. Questo però non significa che sia il linguaggio più richiesto o quello in cui si sceglierebbe oggi di programmare un'applicazione Web ex novo, specie per un'applicazione che deve avere un front-end molto interattivo, dunque basato su Ajax/JSON.

Non che PHP non lo permetta: è un linguaggio general purpose e posso usarlo per programmare ciò che voglio. Non è tuttavia la scelta più ovvia o più adatta a questo tipo di applicazione. E attualmente la combinazione più richiesta per implementare applicativi aziendali su larga scala è quella del linguaggio Java con il framework Spring o Spring Boot.

Tuttavia alla domanda, ha ancora senso studiare PHP oggi? La risposta è **sì**, anche se magari non come primo linguaggio di Web development [specie se siete laureati in informatica! Per i non informatici PHP ha il vantaggio di essere facile da imparare]

# server-side app con Node.js

## uno sguardo introduttivo



# server-side app con Node.JS

## uno sguardo introduttivo

```
import http from "http";
import {IncomingMessage, ServerResponse} from "node:http";

const porta = 8555;

const serverFun = async (req: IncomingMessage, res: ServerResponse) => {
  res.setHeader("Content-Type", "text/html; charset=utf-8");
  res.statusCode = 200;
  res.write(`<h2>${req.method} - ${req.url}</h2>`);
  res.write(`<pre>${JSON.stringify(req.headers)}</pre>`);
  if (req.method === "POST") {
    res.write(`<h3>Body</h3>`);
    const body = await text(req);
    res.write(body);
  }
  res.end();
};

http.createServer(serverFun).listen(porta);
```

## Struttura di una server-side app con il server http di base fornito da Node.js

```
import http from "http";
...
const porta = 8555;
...
http.createServer(serverFun).listen(porta);
```

le librerie di **Node.js** forniscono l'oggetto **http** attraverso cui è possibile creare un server (metodo **createServer**) e attivarlo in ascolto su una data **porta**.

Il metodo **createServer** prende come argomento una funzione che **riceve** le richieste ed **elabora** le risposte, attuando effettivamente le funzionalità dell'applicazione server-side.

```
const serverFun = async (req: IncomingMessage, res: ServerResponse) => {
  ...
};
```

La funzione che passiamo a **createServer** deve prendere come argomenti un oggetto di tipo **IncomingMessage** (la richiesta) e un oggetto **ServerResponse** (la risposta).

# Struttura di una server-side app con il server http di base fornito da Node.js

## IncomingMessage: l'oggetto **request**

Permette di conoscere le caratteristiche della richiesta HTTP.

**req.method**: valore **string** pari a "GET", "POST", ecc...

**req.url**: valore (**string**, contiene la URL **assoluta** quindi senza hostname o port

**req.headers**: oggetto di **coppie chiave-valore**, contiene gli header della richiesta

Se la richiesta è di tipo **POST**, possiamo leggere il **body** utilizzando l'oggetto **req** come **stream di input**<sup>(\*)</sup>. Il modo più semplice per farlo è utilizzare una delle funzioni che Node.js mette a disposizione come **stream consumers**. Queste funzioni «consumano» uno stream di input e restituiscono una **Promise** dell'oggetto in esso contenuto; bisogna usare il **consumer** appropriato per il Content-Type della richiesta. Ad esempio il consumer **text** legge lo stream come testo semplice; il consumer **json** legge lo stream come una JSON string e la deserializza, etc.

## ServerResponse: l'oggetto **response**

Permette di impostare le caratteristiche della risposta HTTP.

**res.setStatus(statusCode)**: specifica lo status code (200, 404, ecc...) della risposta

**res.setHeader(name, val)**: specifica il valore **val** per lo header di nome **name**

Possiamo scrivere il **body** della risposta usando l'oggetto **res** come uno **stream di output**. Il modo più semplice è usare **res.write(text)** per inviare del testo sullo stream, per poi terminarlo con **res.end()**.

Un'altra possibilità è inoltrare un'intero stream di input sullo stream di output:

**req.pipe(res)**

In questo caso non sarà necessario terminare lo stream, termina da solo alla fine dell'inoltro.

(\*) una trattazione approfondita degli stream di Node.js esula dai contenuti del corso; è tuttavia un argomento che va sicuramente approfondito se si intende usare Node.js al di là del piccolo esempio qui presentato.

# server-side app con Node.JS + Express

cenni

```
import express from "express"; // FRAMEWORK
const app = express(); // APP OBJECT, INCLUDES HTTP SERVER
const port = 8883;

// MIDDLEWARE
app.use(express.static("public"));
app.use(express.json());

//REQUEST HANDLERS

app.get("/test/:pr1/:pr2",
    async (req: express.Request, res: express.Response) => {
    ...
})

app.post("/test/:pr1/:pr2",
    async (req: express.Request, res: express.Response) => {
    ...
})

// SERVER ACTIVATION
app.listen(port, () => {
    console.log("Server running on port " + port);
});
```

## Struttura di una server-side app con **Node.js + Express** framework – I

```
import express from "express"; // FRAMEWORK
const app = express(); // APP OBJECT, INCLUDES HTTP SERVER
const porta = 8883;

...

// SERVER ACTIVATION
app.listen(porta, () => {
  console.log("Server running on port " + port);
});
```

In questo caso utilizziamo l'oggetto **app** creato tramite il framework **express**. Questo oggetto include il server http ma fornisce funzionalità aggiuntive.

Una volta configurato l'oggetto **app** possiamo attivarlo in ascolto sulla porta che abbiamo scelto con **app.listen**(port, host?, callback). Se non specifichiamo l'host, per default sarà **localhost**. La **callback** è una funzione che viene eseguita nel momento in cui il server http inizia a ricevere richieste.

## Struttura di una server-side app con Node.js + Express framework – II Middleware

```
...  
// MIDDLEWARE  
app.use(express.static("public"));  
app.use(express.json());  
...
```

In generale il **middleware** è una funzione

(req: **express.Request**, res: **express.Response**, next: **express.NextFunction**) => void

Come già descritto il ruolo del middleware è effettuare elaborazioni preliminari sulla richiesta o sulla risposta, e decidere se la richiesta può «proseguire» il suo iter. In questo caso il middleware invocherà la funzione **next( )** ricevuta come parametro. Se il middleware decide che l'iter **non** deve proseguire, invece di invocare **next( )** potrà inviare una risposta di errore.

Nell'esempio utilizziamo due **middleware** forniti da express: il primo fa sì che i contenuti della cartella indicata vengano serviti **staticamente**. Il secondo **trasforma** la richiesta pre-analizzando il body (se c'è) come JSON string, deserializzandolo e memorizzandolo in una proprietà **body** dell'oggetto **req**.

Si può subordinare l'uso di un **middleware** alla struttura della **route** contenuta nella richieste HTTP:

**app.use(routeExpression, middleware)**

utilizzerà il middleware in questione solo se la url contenuta nella richiesta corrisponde alla routeExpression indicata. La routeExpression è una stringa che esprime una route (o un insieme di route); può includere anche segmenti parametrici preceduti dal carattere : (due punti), e il carattere jolly \* (asterisco)

**/il/mio/pa\*th/:nome/:id**

## Struttura di una server-side app con Node.js + Express framework – III

### Registrazione dei request handlers

```
...  
//REQUEST HANDLERS  
app.get("/test/:pr1/:pr2", handler)  
app.post("/test/:pr1/:pr2", handler)  
...
```

In **express** l'applicazione viene configurata specificando una funzione detta **request handler** per ogni **method+route** dell'API che vogliamo esporre.

L'oggetto **app** mette a disposizione un metodo di registrazione per ogni **method HTTP**. Quindi per gestire le route basate su GET useremo **app.get()**, per quelle basate su POST useremo **app.post()** e così via.

Il primo parametro dei metodi di registrazione è sempre la **route** da gestire. Come per il middleware, anche qui la route può specificare dei segmenti parametrici e contenere il carattere jolly \*. Quindi per essere più corretti dobbiamo dire che l'handler viene associato a un **insieme di route**.

Il **request handler** è una funzione:

(req: **express.Request**, res: **express.Response**) => void

## Struttura di una server-side app con **Node.js + Express** framework – IV

### **express.Request:** l'oggetto **request**

L'oggetto **request** che **express** passa al nostro handler è più **ricco** e **funzionale** del suo corrispettivo di Node.js base, in quanto viene pre-elaborato da **express**. In particolare esso ha delle **proprietà** nelle quali sono già stati estratti i parametri, sia quelli provenienti dalla query string, sia quelli espressi come segmenti parametrici della route. Grazie al middleware è possibile inoltre, come abbiamo già visto, effettuare una pre-lettura dell'eventuale body della richiesta, memorizzando il dato risultante sempre all'interno dell'oggetto **request**.

**req.query:** oggetto di **coppie chiave-valore**, contiene i parametri della query string

**req.params:** oggetto di **coppie chiave-valore**, contiene i segmenti parametrici della route

**req.body:** contiene il body della request, se presente e se interpretato tramite un opportuno middleware

### **express.Response:** l'oggetto **response**

Anche l'oggetto **response** di **express** prevede funzionalità aggiuntive rispetto al suo corrispettivo di base.

Il metodo **send** invia la risposta in un'unica soluzione, curando anche di chiudere lo stream di output al termine dell'invio. Inoltre **rileva automaticamente** le caratteristiche della risposta (come ad es. il **Content-Type** e il **Content-Length**) e imposta di conseguenza i rispettivi **header**.

È inoltre possibile impostare lo status code e inviare la risposta in un'unica linea di codice tramite concatenazione:

```
res.status(code).send(content)
```

## Struttura di una server-side app con **Node.js + Express** framework – V

### Registrazione dei request handlers

```
...  
//REQUEST HANDLERS  
app.get("/test/:pr1/:pr2", handler)  
app.post("/test/:pr1/:pr2", handler)  
...
```

In **express** l'applicazione viene configurata specificando una funzione detta **request handler** per ogni **method+route** dell'API che vogliamo esporre.

L'oggetto **app** mette a disposizione un metodo di registrazione per ogni **method HTTP**. Quindi per gestire le route basate su GET useremo **app.get()**, per quelle basate su POST useremo **app.post()** e così via.

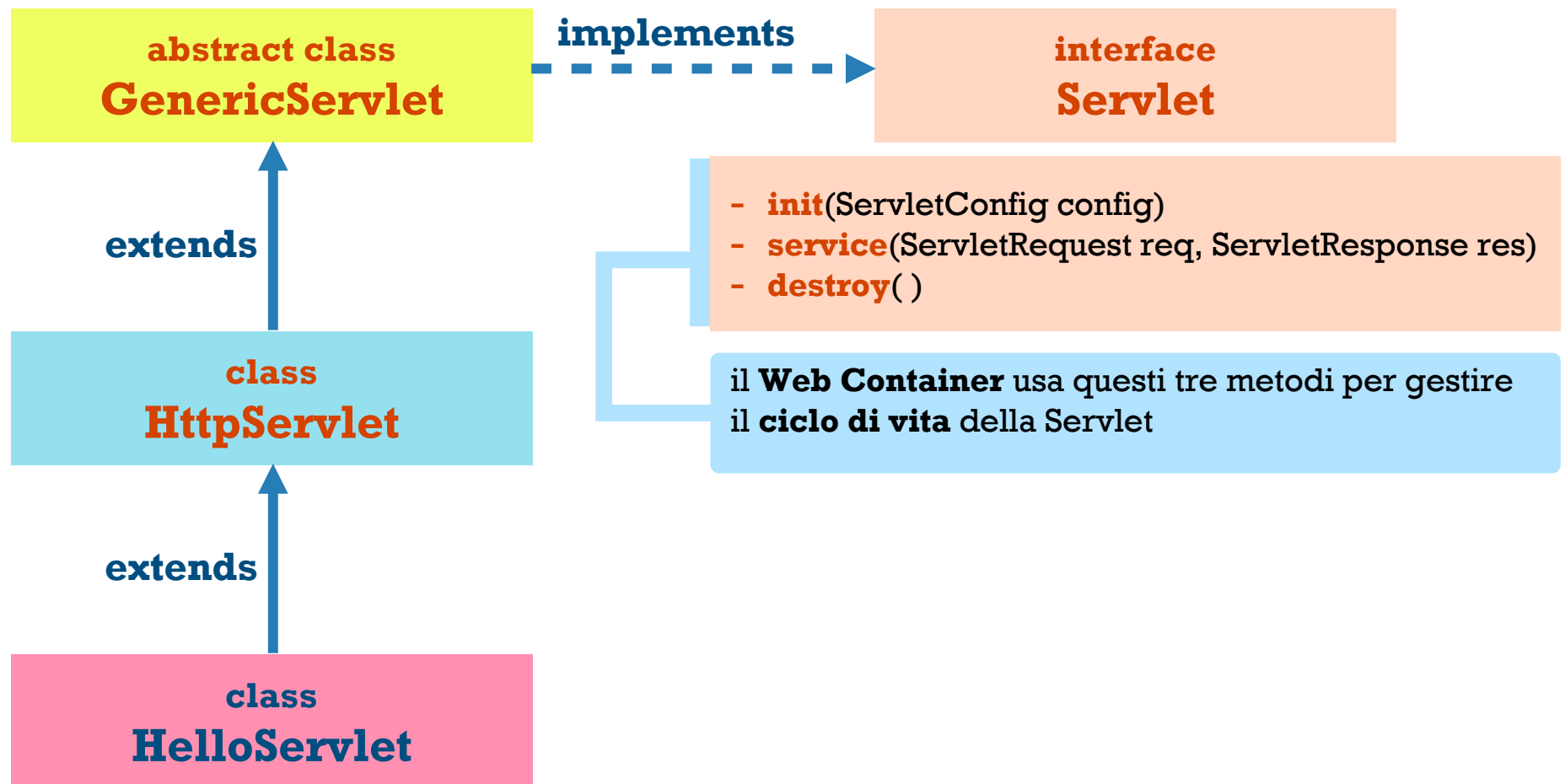
Il primo parametro dei metodi di registrazione è sempre la **route** da gestire. Come per il middleware, anche qui la route può specificare dei segmenti parametrici e contenere il carattere jolly \*. Quindi per essere più corretti dobbiamo dire che l'handler viene associato a un **insieme di route**.

Il **request handler** è una funzione:

(req: **express.Request**, res: **express.Response**) => void

# Gerarchia delle classi

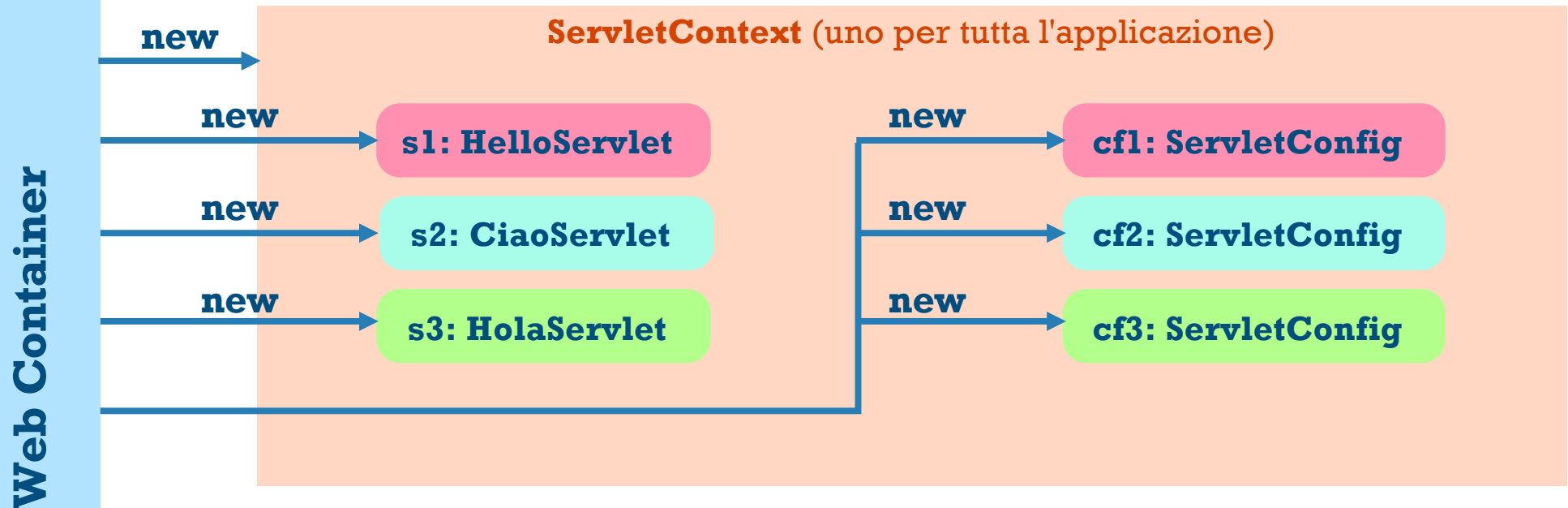
## il cuore della Servlet API



# Ciclo di vita: START

Come il Web Container inizializza l'app

## Fase 1: **creazione istanze**



# Ciclo di vita: START

## Come il Web Container inizializza l'app

### Fase 2: **inizializzazione istanze**

Web Container

**interface**  
**Servlet**

- **init**(ServletConfig config)
- **service**(...)
- **destroy**()

**implements**

**abstract class**  
**GenericServlet**

- **init**(ServletConfig config)
  - memorizza al proprio interno l'oggetto **config**
  - chiama il metodo **init()**
- **init()**
  - non fa nulla ma le Servlet possono ridefinirlo

**s1: HelloServlet**

**cf1**

**s2: CiaoServlet**

**cf2**

**s3: HolaServlet**

**cf3**

**ServletContext**

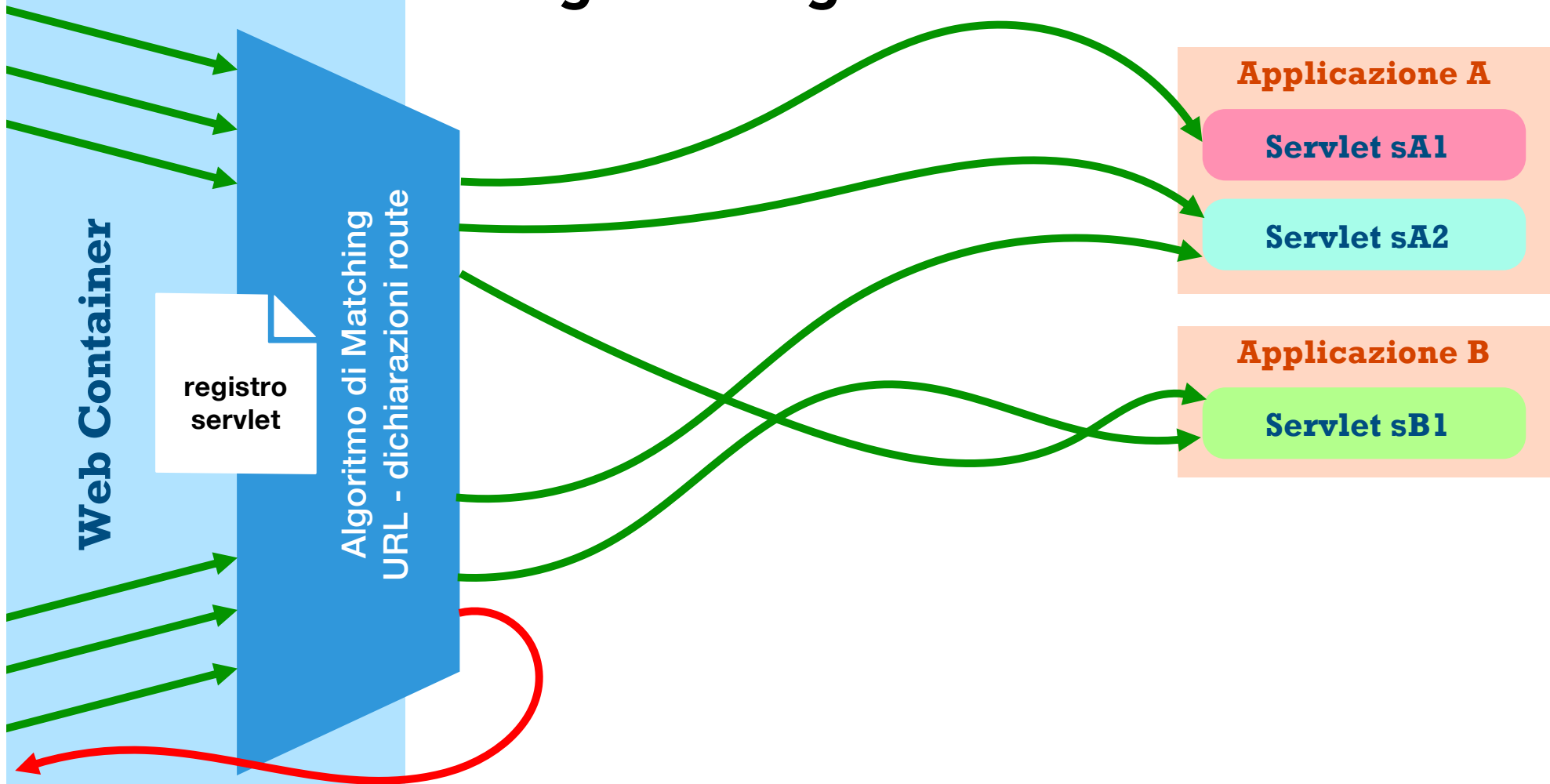
```
public void init() {  
    message = "Hello World!";  
}
```

Solo al completamento della **init** la Servlet è considerata attiva e pronta a ricevere richieste

# Ciclo di vita: SERVICE

## Matching e routing di richieste HTTP

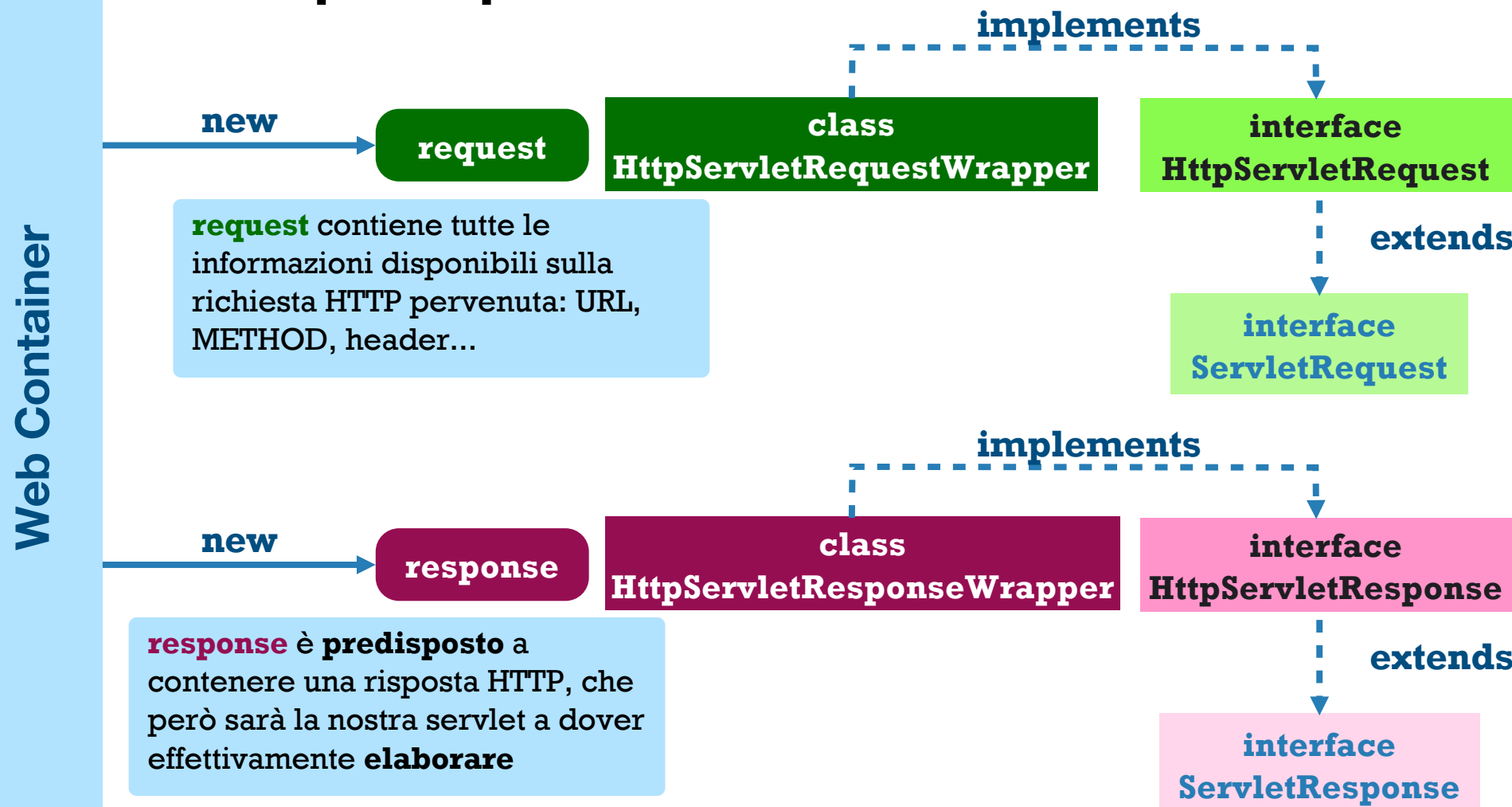
### Fase 1: **matching e routing**



# Ciclo di vita: SERVICE

## Inoltro richieste HTTP

### Fase 2: **predisposizione del servizio**



# Ciclo di vita: SERVICE

## Invocazione del servizio vero e proprio

### Fase 3: **invocazione di service** (della Servlet prescelta)

- se **serv** è la Servlet scelta dal **matching**, il WebContainer chiama  
`serv.service(request, response)`
- **request** e **response** sono gli oggetti **HttpServletRequest** e **HttpServletResponse** creati al passo precedente



**service**(ServletRequest **req**, ServletResponse **res**)

se **req** e **res** sono **HttpServletRequest/Response**, invoca il metodo:

**service**(HttpServletRequest **req**,  
HttpServletResponse **res**)

esamina il **METHOD** della richiesta e, in base ad esso, invoca uno fra:

**doGet**(req, res)  
**doPost**(req, res)  
...etc...

La servlet dovrà implementare uno o più di questi metodi **doXXX** in base a quali tipi di richieste HTTP vuole gestire

# Ciclo di vita: STOP

Come il Web Container dispone dell'app

## Fase 1: **ogni servlet fa le pulizie**

- per ogni **Servlet sN** il WebContainer chiama **sN.destroy()**

**interface**  
**Servlet**

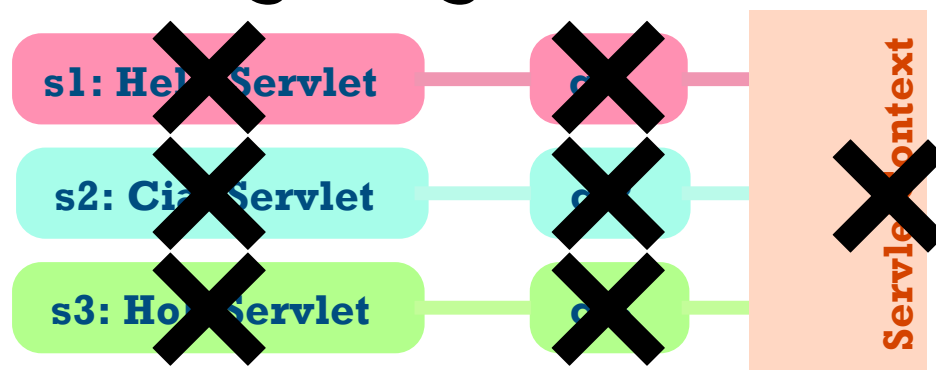
**implements**

**abstract class**  
**GenericServlet**

- **init**(ServletConfig config)
- **service**(...)
- **destroy**()

- **destroy**()
  - non fa nulla
  - può essere **ridefinito** in una Servlet se in fase di **init** abbiamo fatto cose che richiedono di essere disfatte
  - ad es: abbiamo creato una connessione a un DB che deve essere chiusa

## Fase 2: **garbage collection**



## Cos'è una Servlet? - I

Una **Servlet** è un'istanza di una **classe** che **estende** **HttpServlet**.

Jakarta EE specifica una **gerarchia di classi e interfacce** per le Servlet.

Alla base della gerarchia c'è l'**interfaccia Servlet** che definisce quali metodi ogni Servlet deve necessariamente possedere (sono quelli che servono al Web Container per gestirla):

- **init(ServletConfig config)** è il metodo che verrà invocato dal Web Container per inizializzare la Servlet dopo aver creato l'oggetto corrispondente.
- **service(ServletRequest req, ServletResponse res)** è il metodo che verrà invocato dal Web Container quando deve chiedere a una Servlet di gestire una richiesta e ottenere la corrispondente risposta
- **destroy( )** è il metodo che verrà invocato dal Web Container e tramite cui la Servlet può «fare pulizia» di connessioni aperte, servizi sottoscritti, registrazioni effettuate, prima di essere chiusa.

La classe **GenericServlet** offre un'implementazione astratta dell'interfaccia **Servlet** dando un'implementazione di base del metodo **init**: l'oggetto **config** ricevuto viene memorizzato internamente alla Servlet, per poi invocare un metodo astratto **init( )** [senza parametri] che potrà essere implementato dalle sottoclassi.

La classe **HttpServlet** estende **GenericServlet** realizzando le funzionalità di base nel caso in cui la Servlet sia dedicata a richieste HTTP (in linea teorica, è possibile avere Servlet che gestiscono altri tipi di protocolli).

## Cos'è una Servlet? - II

In particolare, la classe **HttpServlet** implementa il metodo **service** di base, mettendo a disposizione delle sue sottoclassi metodi più specifici per gestire le richieste HTTP in base al **method HTTP**

**service(HttpServletRequest req, HttpServletResponse res)**

L'implementazione del metodo **service** di base in **HttpServlet** realizza i seguenti passaggi:

- verifica che la richiesta sia effettivamente una richiesta HTTP
- se è così, crea gli oggetti `HttpServletRequest/Response` a partire dagli originali `ServletRequest/Response`
- invoca una funzione propria, sempre di nome **service**, che però accetta gli `HttpServletRequest/Response`.
- in questa «nuova» **service** esamina il **method HTTP** e a seconda del valore chiama un metodo interno della forma **doXXX** dove XXX è il method (avremo quindi **doGet**, **doPost**, ecc), passando come parametri gli oggetti `HttpServletRequest/Response`
- i metodi **doXXX** non fanno niente: sarà compito delle sottoclassi di `HttpServlet` (dunque, le Servlet che costituiscono l'applicazione) fornire un'implementazione opportuna

## Il ciclo di vita di una servlet – I

Il Web Container usa i metodi dell'interfaccia Servlet (e dunque, indirettamente, la Server API nel suo complesso) per gestire «la vita» di una Servlet. La vita di una Servlet ha inizio nel momento in cui viene installata, oppure (se già installata in precedenza) nel momento in cui il Web Server viene avviato. Si conclude quando viene effettuato lo shut down del Web Server, oppure quando viene disinstallata. Nel mezzo, la Servlet è sempre attiva e pronta a ricevere richieste.

Il ciclo di vita di una Servlet si compone di tre momenti: **START**, **SERVICE** e **STOP**. Ciascuno di essi può essere diviso in sotto-fasi.

### **START**

#### **1 – Creazione**

- il Web Container crea un **ServletContext** che sarà comune a tutta l'applicazione
- crea una singola istanza **sN** per ciascuna Servlet contenuta nell'app
- per ogni Servlet, crea un oggetto **ServletConfig cfN** contenente i parametri di configurazione della Servlet (usando, fra le altre cose, le informazioni annotate con `@WebServlet`) e contenente un riferimento al **ServletContext**.
- inserisce in un suo **registry** le Servlet e le **route** a cui ciascuna risponde

#### **2 – Inizializzazione**

- per ogni Servlet **sN** con corrispondente ServletConfig **cfN**, il WebContainer chiama **sN.init(cfN)**, il quale invoca a sua volta il metodo **sN.init( )**

# Il ciclo di vita di una servlet – II

## SERVICE

### 1 – Matching & Routing

- il Web Container effettua un **matching** fra la URL ricevuta e gli **urlPatterns** dichiarati dalle Servlet determinando la servlet «vincente» **s**
- l'algoritmo ha una strategia per dirimere i casi di **conflitto** (ad esempio, se una URL corrisponde a due pattern, "vince" quello più lungo)
- se non c'è matching: **404 not found**

### 2 – Predisposizione

- vengono predisposti gli oggetti **HttpServletRequest/Response** che estendono gli oggetti ServletRequest/Response di partenza

### 3 – Invocazione del servizio

- viene invocato **s.service(req, res)** di **HttpServlet**, che riceve come parametri gli oggetti **HttpServletRequest/Response** precedentemente predisposti
- a sua volta esso invocherà il metodo **s.doXXX** appropriato per la richiesta HTTP in esame

## STOP

### 1 – Cleaning up

- per ogni servlet **sN**, il Web Container chiama il metodo **sN.destroy()**

### 2 – Garbage collection

- vengono eliminati gli oggetti Servlet, gli oggetti ServletConfig, e l'oggetto ServletContext